



Security Assessment



Item 11 Internal Report

August 2026

Prepared for ether.fi



Disclaimer

This document is intended for **internal** use only.



Initial Commit Hash	Final Commit Hash
ccb8fe30f3	dd7b4cac74

Protocol Scope

- src/modules/cctp/CCTPModule.sol

Informational Issues

I-01. Signed bridge requests do not bind fee or messenger configuration

Description: The signed digest commits only to `BridgeParams`. It does not commit to the selected `TokenMessenger`, provider-fee rate or recipient and CCTP maximum fee. These values are loaded from current administrative configuration when `requestBridge` is submitted and are only snapshotted afterward.

Consequently, a relayer can retain a valid signature until configuration becomes less favorable and then submit it using the same signed amount. Fees can rise from the value quoted when signing to a 5% provider fee plus a CCTP fee of up to 5% of the remaining amount, reducing destination proceeds by as much as 9.75%. The post-request snapshot does not protect against configuration changes between off-chain signing and on-chain submission.

Furthermore, if the withdrawal delay is zero, the request can be executed instantly, preventing the user from cancelling a bad request.

Recommendation: Consider snapshotting the fee or acknowledging this issue.

Customer's response: Acknowledged



I-02. Global domain allowlist accepts unsupported asset-domain routes that can strand burned tokens

Description: Asset support and destination-domain support are configured independently. Consequently, `requestBridge` accepts the product of every configured asset and every allowed domain.

CCTP destination token mappings are route-specific. A remote TokenMessenger may exist for a domain while a particular burn token has no corresponding mint token configured there. In that case, the source `depositForBurn` can successfully burn the tokens, but destination message execution cannot mint them.

Enabling both asset A on domain X and Y, and asset B only on X will also incorrectly enable B-to-Y. Such a message remains unmintable unless Circle later configures that destination token mapping.

Recommendation: Consider snapshotting the fee or acknowledging this issue.

Customer's response: Fixed in [a6e54ba8aa](#)

Fix review: Fix confirmed

I-03. EVM-specific address casting prevents bridging CCTP tokens to non-EVM chains

Description: In `CCTPModule.sol` and `SettlementDispatcherV2.sol`, the destination recipient address is defined as a standard 20-byte EVM `address` and is cast to a `bytes32` for the CCTP payload by zero-padding. Circle's CCTP supports bridging to non-EVM chains (such as Solana), which require full 32-byte addresses.

Because the contract interface forces the user to input a 20-byte `address`, it is impossible to specify a valid non-EVM recipient. Bridging to such domains mints to an address nobody controls, resulting in a total loss of the burned amount, while request-time validation (`destRecipient != address(0)`) cannot detect it.

Recommendation: Consider acknowledging this issue, and documenting this behaviour.

Customer's response: Fixed in [a6e54ba8aa](#)

Fix review: Fix confirmed



I-04. Orphaned withdrawals[safe] records in the zero-delay path / silent overwrite of a queued bridge

Description: `requestBridge` never checks whether a bridge is already queued for the safe, and in the zero-delay branch it neither stores nor clears `withdrawals[safe]`. If the withdrawal delay is reduced to zero while a bridge is queued, the immediate burn executes but the stale record for the previous request remains in storage indefinitely.

Recommendation: Consider revering or explicitly cancelling when `withdrawals[safe].destRecipient != address(0)` at the start of `requestBridge`, and execute `delete withdrawals[safe]` on the zero-delay path.

Customer's response: Fixed in [a6e54ba8aa](#)

Fix review: Fix confirmed

I-05. CCTPModule fee views return valid-looking quotes for unsupported assets and unvalidated finality modes

Description: `_setAssetConfigs` writes the whole `AssetConfig` struct unconditionally, allowing out-of-range fee values to persist. `getProviderFee` never checks `tokenMessenger` and returns fees for unsupported assets. `getBridgeFee` returns the ETH sentinel as `feeToken` with zero fees for unsupported assets, leading consumers to build transactions that revert.

Recommendation: Validate `maxFeeBps / providerFeeBps` unconditionally in `_setAssetConfigs`. Make `getproviderFee` and `getBridgeFee` revert with `UnsupportedAsset` or return an explicit `bool supported` flag.

Customer's response: Fixed in [a6e54ba8aa](#)

Fix review: Fix confirmed



I-06. requestBridge never validates the amount against CCTP's per-message burn limit

Description: `requestBridge` never reads the configured messenger's minter for `burnLimitsPerMessage(asset)` nor enforces any per-request cap of its own. A bridge whose `burnAmount` exceeds the live burn limit is accepted, queues a CashModule withdrawal, and reserves `p.amount`. `executeBridge` then reverts inside `depositForBurn` on every attempt, leaving the amount reserved until unwound.

Recommendation: Consider querying the configured messenger's minter for `burnLimitsPerMessage(asset)` or storing an admin-configured `maxBridgeAmount` per asset, and rejecting oversized `burnAmount` at `requestBridge` time. Alternatively, consider acknowledging this issue.

Customer's response: Fixed in [a6e54ba8aa](#)

Fix review: Fix confirmed

I-07. Deleting withdrawals does not follow CEI pattern

Description: Withdrawals within the `CCTPModule` are deleted after the withdrawal is processed in `CashModule` and after the call to `_bridge`, which breaks checks-effects-interactions pattern. Other recent modules delete from storage before making external calls to the cash module or external protocols.

Recommendation: Delete `withdrawals[safe]` prior to calling `cashModule.processWithdrawal[safe]` and `_bridge`.

Customer's response: Fixed in [a6e54ba8aa](#)

Fix review: Fix confirmed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.